

AERO-GUARD • AUTONOMOUS SURVEILLANCE ECOSYSTEM

System Architecture & Technical Manual

Comprehensive Guide: WebSockets, YOLO Vision Models, Re-ID, SQLite WAL & Tactical HUD

1. Executive Summary & System Overview

AERO-GUARD is a real-time, decoupled autonomous aerial drone and perimeter security surveillance system. It unifies YOLO deep learning computer vision, multi-object tracking (ByteTrack / BoT-SORT), cross-camera Person Re-Identification (X-TFCLIP), restricted geofence perimeter monitoring, and a military-grade Next.js 16 Tactical HUD.

Key Architectural Pillars

- Dual-Perspective Vision:** Seamlessly switches between top-down Aerial UAV perspective and horizontal Ground CCTV perimeter perspective.
- Sub-50ms Latency:** Hardware-accelerated inference running at 25-30 FPS with live WebSockets telemetry streaming.
- Cross-Camera Re-ID:** Re-identifies suspects across different camera angles and lighting conditions using vision-language transformer embeddings.
- Edge-Native Evidence Locker:** In-process SQLite database with Write-Ahead Logging (WAL) and automated incident recording.
- Zero Code Duplication:** Strict senior-level engineering adherence with centralized schemas, unified types, and decoupled services.

Core Technology Stack

Technology Layer	Libraries & Frameworks
Computer Vision Engine	Ultralytics YOLOv8 / YOLO11 / YOLO26, ByteTrack, BoT-SORT, OpenCV
Cross-Camera Re-ID	X-TFCLIP Transformer, PyTorch, Torchvision, Cosine Similarity Ranking
Backend Microservice	FastAPI (Python 3.12), Starlette, Uvicorn (ASGI), Pydantic v2
Real-Time Protocols	WebSockets (Full-Duplex Telemetry/Alarms), Multipart MJPEG (Video Stream)
Database & Storage	SQLite 3 with WAL Journal Mode, H.264 MP4 Container Writer, JPEG Evidence
Frontend Tactical HUD	Next.js 16 (Turbopack), React 19, TypeScript 5, Tailwind CSS, Redux Toolkit, TanStack Query
Audio Synthesizer	Web Audio API Procedural Tactical Siren Synthesizer
CI/CD Pipeline	GitHub Actions (.github/workflows/ci.yml) with strict Python & TypeScript quality gates

2. Real-Time WebSockets Architecture & Data Flow

WebSockets establish a persistent, bidirectional, full-duplex TCP connection between the FastAPI backend and Next.js frontend, enabling instantaneous telemetry synchronization and zero-latency intrusion alerts without HTTP polling overhead.

Architectural Rationale: Why WebSockets?

- **Why WebSockets over HTTP Polling?:** Traditional HTTP polling sends repeated HTTP requests every few milliseconds, incurring high HTTP header overhead, TCP connection churn, and 200-500ms latency spikes. WebSockets maintain a single persistent TCP socket, streaming JSON payloads in under 5 milliseconds.
- **Why WebSockets over Server-Sent Events (SSE)?:** While SSE is unidirectional (server-to-client only), WebSockets provide full-duplex communication, allowing the tactical HUD to both receive 30 FPS telemetry and dispatch instant commands over the same channel infrastructure.
- **Channel Multiplexing:** The system uses isolated WebSocket channels for `aerial` (drone avionics + top-down detections), `ground` (CCTV perimeter detections + gate controls), and `alerts` (instant intrusion broadcast).

Source Code Implementation Files

File Path	Responsibility	Technical Details
backend/app/api/v1/endpoints/stream.py	FastAPI WebSocket Endpoints	Defines `@router.websocket('/ws/telemetry')` and `'/ws/alerts'`. Manages active client connection sets, parses channel query parameters (`?channel=aerial`), and safely handles client disconnects.
backend/app/services/streaming/drone_service.py	Avionics Broadcast Loop	Generates real-time 6S LiPo battery discharge rates, RTK GPS fixes, altitude, heading physics, and pushes telemetry state to connected WebSocket listeners at 10 Hz.
backend/app/services/streaming/telemetry_state.py	Thread-Safe State Cache	Stores the latest detection metrics, active track IDs, threat levels, and FPS benchmarks in memory to serve immediate snapshots to new WebSocket connections.
frontend/src/hooks/useTelemetrySocket.ts	Next.js Telemetry Hook	Maintains active WebSocket connection to `ws://localhost:8000/ws/telemetry?channel={perspective}`. Dispatches incoming bounding boxes and drone metrics directly into Redux Toolkit state.
frontend/src/hooks/useAlertSocket.ts	Next.js Alert Hook	Dedicated listener for high-priority intrusion events. Triggers procedural audio sirens and alert banners the exact millisecond a geofence breach occurs.
frontend/src/services/telemetryWs.ts	WebSocket Client Service	Encapsulates WebSocket connection lifecycle, heartbeat keep-alives, and exponential backoff auto-reconnection logic.

3. AI Deep Learning Vision Models & Person Re-Identification

AERO-GUARD employs specialized neural network architectures tailored specifically for overhead UAV surveillance, eye-level perimeter CCTV monitoring, multi-object tracking, and cross-camera suspect re-identification.

AI Models & Deep Learning Checkpoints

Model	Role	Task & Dataset	File Path
visdrone_person_best.pt	Aerial Drone Model	YOLO11n fine-tuned on the VisDrone-DET dataset. Specifically optimized for high-altitude UAV perspectives, small-scale person bodies (10-30 pixels), steep camera pitch angles, and aerial motion blur.	backend/models/visdrone_person_best.pt
yolo26n.pt / yolov8n.pt	Ground CCTV Model	YOLO26n/YOLOv8n trained on COCO dataset. Optimized for horizontal eye-level perimeter cameras, full human body silhouettes, close-to-medium range surveillance, and low-light night conditions.	backend/models/yolo26n.pt

ByteTrack / BoT-SORT	Multi-Object Trackers	Kalman-filter and Hungarian-matching algorithms. Tracks person trajectories across video frames, maintains persistent Track IDs, and recovers identities after short-term occlusions.	backend/app/services/detector/ & tracking/
X-TFCLIP (xtfclip.pth.tar)	Cross-Camera Re-ID	Vision-Language Transformer generating 512-dimensional embedding vectors from cropped suspect images. Computes cosine similarity between Ground query suspects and Aerial drone candidate detections.	backend/app/services/reid/ & backend/models/
mot20_yolo26s_pedestrian.pt	Crowd Density Model	Specialized pedestrian detector checkpoint for dense crowd surveillance and high-occupancy perimeter zones.	backend/models/mot20_yolo26s_pedestrian.pt

Source Code Implementation Files

File Path	Responsibility	Technical Details
backend/app/services/detector/detector.py	Inference & Tracking Engine	Executes forward pass through YOLO models using PyTorch, filters bounding boxes by confidence threshold, applies Non-Maximum Suppression (NMS), and associates detections via ByteTrack.
backend/app/services/inference/dispatcher.py	Dynamic Model Dispatcher	Provides lazy-loading and dynamic hot-swapping between `aerial` (VisDrone) and `ground` (COCO) models at runtime without restarting the server.
backend/app/services/inference/model_loader.py	Model Integrity & Checksums	Verifies file existence and validates SHA-256 cryptographic hashes against `backend/models/registry.json` before loading weights into memory.
backend/app/services/inference/profiles.py	Perspective Profiles	Defines recommended image resolutions (640x640 vs 1280x1280), default confidence thresholds (0.45 vs 0.50), and NMS IOU ratios for each camera mode.
backend/app/services/reid/encoder.py	X-TFCLIP Feature Extractor	Normalizes suspect bounding box crops to 224x224 and runs the X-TFCLIP transformer backbone to extract high-dimensional semantic feature embeddings.
backend/app/services/reid/service.py	Re-ID Matching Service	Maintains gallery of detected person tracklets, samples high-quality crops, and calculates Cosine Similarity scores to rank matching suspects.

4. Database Architecture & SQLite Write-Ahead Logging (WAL)

AERO-GUARD uses an embedded SQLite database engine configured with Write-Ahead Logging (WAL) mode to record incident metadata, suspect snapshot paths, and video recordings.

Why SQLite with WAL Mode Over PostgreSQL / MySQL / MongoDB?

- **1. Zero Network Latency & In-Process C-Speed:** Computer vision pipelines process frames at 25-30 FPS (33ms per frame). A traditional client-server database (PostgreSQL or MySQL) introduces network socket round-trips, connection pool overhead, and serialization latency (5-20ms per query). SQLite runs directly in the Python process memory via C-level bindings with microsecond query execution times.
- **2. Zero Operational Footprint & Edge/Drone Native:** AERO-GUARD is designed for tactical edge deployments (field laptops, mobile command vehicles, drone companion computers like NVIDIA Jetson). Running a separate PostgreSQL or

MongoDB daemon consumes valuable RAM and requires background database service management. SQLite requires zero background services.

- **3. WAL Mode Non-Blocking Concurrency:** In standard SQLite rollback-journal mode, database writes lock the entire file, blocking readers. With Write-Ahead Logging (`PRAGMA journal_mode=WAL;`), writers write append-only logs while readers read simultaneously from snapshot views. This allows high-speed video inference writes without blocking REST API or WebSocket queries.
- **4. Forensic Chain of Custody & Single-File Portability:** In security and defense applications, incident data must often be preserved for legal or forensic analysis. The entire database is contained in a single self-contained file (`runs/output/evidence.db`), allowing effortless one-click backups, USB export, or air-gapped forensic transfers.
- **5. Hybrid Storage Architecture:** Relational metadata (timestamps, track IDs, threat levels, zone violations) is indexed in SQLite tables, while heavy binary files (JPEG high-res snapshots, MP4 incident video recordings) are saved directly to the filesystem with relative path pointers. This keeps database size compact and query execution instantaneous.

Source Code Implementation Files

File Path	Responsibility	Technical Details
backend/app/db/connection.py	Connection Pool & WAL Initializer	Creates SQLite connections, enables <code>PRAGMA journal_mode=WAL;</code> , <code>PRAGMA synchronous=NORMAL;</code> , and <code>PRAGMA foreign_keys=ON;</code> .
backend/app/db/schema.py	Database DDL Definitions	Defines the <code>evidence_records</code> table, timestamp columns, threat level enums, JSON metadata columns, and optimized B-Tree indexes on <code>created_at</code> and <code>threat_level</code> .
backend/app/db/repository.py	Data Access Repository (DAO)	Provides clean high-level CRUD methods: <code>create_record()</code> , <code>get_records()</code> , <code>get_record_by_id()</code> , <code>delete_record()</code> , and <code>get_statistics()</code> .
backend/app/db/query_builder.py	Parameterized Query Builder	Constructs safe, SQL-injection-proof parameterized SQL statements with dynamic filtering by threat level, perspective, date range, and pagination offsets.
backend/app/db/mappers.py	Row-to-Model Data Mappers	Safely converts raw SQLite tuple rows into typed <code>EvidenceRecord</code> Pydantic models with robust JSON parsing fallbacks.
backend/app/db/syncer.py	Storage Reconciler & Syncer	Scans physical <code>runs/output/snapshots/</code> and <code>recordings/</code> directories to ensure physical files match database rows, pruning orphaned records.

5. Configuration Files, YAML Workflows & Environment Setup

AERO-GUARD relies on declarative configuration files for automated CI/CD testing, environment variable loading, and frontend build optimization.

Configuration & CI/CD Files

File Name	Scope	Purpose & Key Details
-----------	-------	-----------------------

<code>.github/workflows/ci.yml</code>	GitHub Actions CI Pipeline	Automated Continuous Integration workflow running on every Git push and pull request. Contains two parallel jobs: <code>`backend-ci`</code> (Python 3.12, dependency caching, bytecode verification, model downloads, 118 unit tests, synthetic video generator test, headless detection pipeline test, and FastAPI microservice health check) and <code>`frontend-ci`</code> (Node 20.x, npm ci, TypeScript strict typecheck <code>`npx tsc --noEmit`</code> , ESLint quality check, and Next.js production build <code>`npm run build`</code>).
<code>backend/.env</code>	Backend Environment Variables	Local environment overrides: <code>`DETECTION_DEVICE=cpu`</code> (or <code>`cuda`</code>), <code>`DEFAULT_MODEL_AERIAL=visdrone_person_best.pt`</code> , <code>`DEFAULT_MODEL_GROUND=yolo26n.pt`</code> , <code>`VIDEO_END_BEHAVIOR=loop`</code> (enables seamless video looping), <code>`REID_ENABLED=false`</code> , <code>`CORS_ORIGINS=*`</code> .
<code>backend/app/core/config.py</code>	Backend Settings & Dataclasses	Type-safe dataclass configuration classes: <code>`DetectionConfig`</code> , <code>`ReIDConfig`</code> , <code>`StreamingConfig`</code> , and default threshold boundaries.
<code>backend/app/core/constants.py</code>	Global System Constants	Immutable system constants: default geofence polygon vertices, model SHA-256 hashes, stream buffer limits, and threat level definitions.
<code>backend/app/core/device.py</code>	Hardware Acceleration Resolver	Automatically detects host hardware (NVIDIA CUDA GPU vs. Apple Silicon / CPU) and configures PyTorch device dispatch safely.
<code>frontend/package.json</code>	Frontend Package Manifest	Defines Next.js 16, React 19, TypeScript, Tailwind CSS, Redux Toolkit, TanStack Query, and Lucide React dependencies.
<code>frontend/tsconfig.json</code>	TypeScript Compiler Options	Enforces strict TypeScript compilation, <code>`@/*`</code> path alias mapping to <code>`./src/*`</code> , and modern ES2022 module resolution.
<code>frontend/tailwind.config.ts</code>	Tailwind CSS Configuration	Configures tactical dark-mode color palettes (zinc/slate/emerald/amber/rose), scanline animations, and glassmorphic backdrop filters.
<code>frontend/next.config.ts</code>	Next.js Build Configuration	Configures Turbopack bundler options, React strict mode, and API proxy rewrites.

6. Complete Folder & Component Directory Breakdown

Detailed breakdown of every directory and file across the entire repository.

Directory Manifest & Submodules

Directory: `backend/app/api/v1/`

Purpose: REST API endpoints and WebSocket servers connecting the tactical frontend HUD to the vision pipeline. | **Tech:** FastAPI, Starlette, WebSockets, Pydantic v2

File Name	Responsibility & Details
<code>router.py</code>	Master API router mounting all sub-endpoints under the <code>`/api`</code> prefix.
<code>endpoints/stream.py</code>	MJPEG video streaming (<code>`/api/stream/video_feed`</code>), stream source switching, and video file upload endpoint.
<code>endpoints/ws.py</code>	WebSocket endpoint (<code>`/ws/telemetry`</code>) streaming high-speed bounding boxes and avionics.
<code>endpoints/cameras.py</code>	Tactical camera fleet registration, RTSP link updating, and network connection probing.
<code>endpoints/drone.py</code>	Drone flight directives (<code>`takeoff`</code> , <code>`patrol`</code> , <code>`rtl`</code>) and ground perimeter commands (<code>`preset_gate`</code> , <code>`preset_patrol`</code> , <code>`ir_filter`</code> , <code>`reboot_sensor`</code>).
<code>endpoints/reid.py</code>	Cross-camera Person Re-ID status queries and manual matching retries.

<code>endpoints/config.py</code>	Live surveillance configuration tuning (confidence thresholds and restricted zone polygons).
<code>endpoints/alerts.py</code>	Incident alert logs and threat level status evaluations.
<code>endpoints/snapshots.py</code>	Intruder evidence photo capture and gallery retrieval.
<code>endpoints/recordings.py</code>	Video clip recording archive queries and downloads.
<code>endpoints/evidence.py</code>	Paginated SQLite database search and deletion for recorded media.
<code>endpoints/tracking.py</code>	Tracking mode switching (automatic multi-person vs manual suspect lock).
<code>endpoints/generative.py</code>	Cloud generative aerial diffusion image synthesis via Replicate API.

Directory: `backend/app/core/`

Purpose: Centralized configuration, paths, hardware device detection, and environment variables. | **Tech:** Python
Dataclasses, PyTorch Device Detection, OS Paths

File Name	Responsibility & Details
<code>config.py</code>	Dataclass configs (<code>DetectionConfig</code> , <code>ReIDConfig</code>), threshold limits, and video playback mode (<code>loop</code>).
<code>constants.py</code>	Global constants, default geofence coordinates, and model checksums (SHA-256).
<code>paths.py</code>	Resolves filesystem locations for <code>runs/output/</code> , <code>evidence.db</code> , <code>snapshots/</code> , <code>recordings/</code> , and <code>uploads/</code> .
<code>device.py</code>	Selects optimal compute device (<code>cuda</code> on NVIDIA GPUs vs. safe <code>cpu</code> fallback on macOS/Linux).
<code>env.py</code>	Safely loads environment variables from <code>backend/.env</code> with defaults.

Directory: `backend/app/db/`

Purpose: SQLite database engine, connection pooling, table schemas, and decoupled data access repositories. | **Tech:**
SQLite 3 (WAL Mode), Python `sqlite3`, Parameterized SQL

File Name	Responsibility & Details
<code>connection.py</code>	Thread-safe SQLite connection manager with WAL mode for non-blocking concurrent reads/writes.
<code>schema.py</code>	DDL table definitions for <code>evidence_records</code> and query optimization indexes.
<code>repository.py</code>	CRUD data-access repository executing inserts, paginated queries, and file deletion.
<code>mappers.py</code>	Converts SQLite rows into typed <code>EvidenceRecord</code> Pydantic models with safe JSON parsing.
<code>query_builder.py</code>	Constructs safe parameterized SQL queries for filtering by threat level, perspective, and date.
<code>syncer.py</code>	Scans physical snapshot and recording directories to ensure disk files match database rows.

Directory: `backend/app/schemas/`

Purpose: Pydantic data models enforcing strict request/response data contracts. | **Tech:** Pydantic v2

File Name	Responsibility & Details
<code>evidence.py</code>	Data models for database evidence items (<code>EvidenceRecord</code> , <code>EvidenceListResponse</code>).
<code>drone.py</code>	Flight directives (<code>DroneCommandRequest</code>) and avionics telemetry (<code>DroneAvionicsData</code>).
<code>config.py</code>	Payloads for confidence thresholds, zone polygons, and stream sources.
<code>cameras.py</code>	Camera registration, RTSP link definitions, and connection diagnostics models.
<code>reid.py</code>	Suspect matching targets, candidates, and similarity score schemas.

tracking.py	Tracking mode configurations (auto vs. manual suspect locking).
generative.py	Aerial diffusion prompt requests and synthesized image outputs.

Directory: backend/app/services/detector/

Purpose: Real-time YOLO deep learning inference and multi-object tracking. | **Tech:** Ultralytics YOLO, PyTorch, ByteTrack, BoT-SORT

File Name	Responsibility & Details
detector.py	Executes YOLOv8/YOLO11 inference, applies confidence/IOU thresholds, and tracks movement vectors.
profiler.py	Real-time FPS calculation profiler.

Directory: backend/app/services/inference/

Purpose: Model weight loading, integrity checking, and perspective profile switching. | **Tech:** PyTorch, Torchvision, JSON Registry, Cryptography

File Name	Responsibility & Details
dispatcher.py	Lazy loading and dynamic model switching between `aerial` (VisDrone) and `ground` (COCO).
profiles.py	Defines model profiles, recommended image sizes, confidence, and tracker configs.
model_loader.py	Verifies file existence and SHA-256 checksums before loading model weights.
registry_loader.py	Loads dynamic model overrides from `backend/models/registry.json`.
status_reporter.py	Builds detailed diagnostic status reports on loaded AI models.

Directory: backend/app/services/streaming/

Purpose: Live video frame ingestion, MJPEG broadcasting, and multi-camera channel coordination. | **Tech:** OpenCV (cv2), Starlette StreamingResponse, Threading

File Name	Responsibility & Details
stream_coordinator.py	Master stream manager linking camera sources to the MJPEG broadcaster.
channels.py	Manages concurrent independent camera streams for Aerial Drone and Ground CCTV.
frame_streamer.py	Encodes tactical HUD-annotated frames into live multipart JPEG bytes.
pipeline_processor/	Modular pipeline processing frames through detection, zoning, and alerts.
sources/	Stream source ingestion for `synthetic`, `webcam`, `file` (MP4 video), and `rtsp` (POE CCTV).
drone_service.py	Simulates drone avionics: 6S LiPo battery, RTK GPS, altitude, and heading physics.
connection_prober.py	Tests network reachability and port latency for RTSP camera feeds.
telemetry_state.py	Thread-safe in-memory cache of the latest detection metrics and threat statuses.

Directory: backend/app/services/reid/

Purpose: Cross-camera Ground-to-Aerial Person Re-Identification using transformer embeddings. | **Tech:** X-TFCLIP Transformer, PyTorch, Torchvision, Cosine Similarity

File Name	Responsibility & Details
service.py	Manages suspect crop sampling, query sequences, and cosine similarity ranking.
encoder.py	Loads X-TFCLIP transformer model and extracts 512-dimensional visual embedding vectors.

crops.py	Crops, resizes, and normalizes detected person bounding boxes for feature extraction.
worker.py	Runs heavy transformer inference in an isolated background process to prevent stream lag.
tracklets.py	Data structures tracking rolling sequences of suspect crops across frames.
assets.py	Handles downloading and integrity verification of `xtfclip.pth.tar` weights.

Directory: backend/app/services/zone/

Purpose: Restricted perimeter geofencing and point-in-polygon intrusion math. | **Tech:** OpenCV (cv2.pointPolygonTest), Vector Geometry

File Name	Responsibility & Details
zone_monitor.py	Checks person foot coordinates against polygon geofences using `cv2.pointPolygonTest`.
gathering.py	Detects high-density clusters of people gathering in close proximity.

Directory: backend/app/services/alert/

Purpose: Threat level classification, alert debouncing, and automated photo snapshot evidence capture. | **Tech:** OpenCV (cv2.imwrite), Python Threading

File Name	Responsibility & Details
alert_manager.py	Evaluates threat tiers (`CLEAR`, `MONITORING`, `INTRUSION`, `MANUAL`) and alert messages.
evidence_recorder.py	Saves high-resolution intruder snapshot photos and registers database rows.
state_evaluator.py	Manages alert cooldown timers to prevent duplicate snapshot spam.

Directory: backend/app/services/recording/

Purpose: Incident video clip recording with rolling pre-roll memory buffering. | **Tech:** OpenCV VideoWriter, Collections Deque, Threading

File Name	Responsibility & Details
recorder.py	Coordinates incident video clip recording triggers and durations.
preroll_buffer.py	Circular memory buffer keeping the previous 5 seconds of footage prior to alarm.
container_writer.py	Asynchronously writes `.mp4` video files to `runs/output/recordings/`.
persistence.py	Registers completed recording metadata and video durations in SQLite.

Directory: backend/app/simulator/

Purpose: Procedural synthetic drone flight and pedestrian video simulation generator. | **Tech:** OpenCV Drawing, NumPy Trajectory Math

File Name	Responsibility & Details
video_generator.py	Generates synthetic `.mp4` test videos with moving pedestrians and drone telemetry.
person.py	Simulates realistic walking trajectories, speeds, and body proportions.
persons/ground_physics.py	Eye-level pedestrian physics simulation for CCTV camera perspectives.

Directory: backend/models/

Purpose: Trained deep learning model weights and catalog registry. | **Tech:** PyTorch Checkpoints (.pt / .tar), JSON Catalog

File Name	Responsibility & Details
-----------	--------------------------

visdrone_person_best.pt	Aerial Drone Model: YOLO11n fine-tuned on VisDrone for overhead person detection.
yolo26n.pt	Ground CCTV Model: YOLO26n optimized for eye-level person detection on CCTV/webcams.
mot20_yolo26s_pedestrian.pt	High-accuracy pedestrian tracking checkpoint for crowded scenes.
registry.json	Catalog defining model filenames, input resolutions, confidence thresholds, and download URLs.

Directory: frontend/src/app/

Purpose: Next.js 16 App Router pages and navigation deep links. | **Tech:** Next.js 16, React 19, TypeScript, Turbopack

File Name	Responsibility & Details
layout.tsx	Root layout wrapping all pages in Redux, React Query, WebSockets, and global styles.
page.tsx	Default root route ('/') redirecting to the mission command dashboard.
airspace/page.tsx	Deep-link page for Aerial Drone Airspace Command view ('/airspace').
cameras/page.tsx	Deep-link page for Perimeter CCTV Camera Wall view ('/cameras').
incidents/page.tsx	Deep-link page for Real-Time Incident Logs and Alarms ('/incidents').
recordings/page.tsx	Deep-link page for Evidence Database and Video Playback ('/recordings').
settings/page.tsx	Deep-link page for Detection Calibration and Geofence Tuning ('/settings').
globals.css	Tailwind directives, military scanline animations, and dark mode tokens.

Directory: frontend/src/components/tactical/

Purpose: Tactical HUD widgets, video viewports, control decks, and modal overlays. | **Tech:** React 19, Tailwind CSS, Lucide React, Glassmorphism

File Name	Responsibility & Details
VideoViewport/	Live MJPEG video screen, split grids (1-UP, 2-UP, 4-UP), suspect cards, and upload controls.
Header/	Tactical header with threat ribbon, UTC military clock, and AI model status.
TacticalSidebar/	Collapsible left navigation bar linking to all command views.
FlightControlDeck/	Drone flight directive buttons (`Takeoff`, `Patrol`, `RTL`, `Land`).
PerimeterSecurityDeck/	Ground CCTV controls (`Lock Gate`, `Patrol 360`, `IR Filter`, `Reset PTZ`).
DroneAvionicsCard/	Telemetry tile showing 6S LiPo battery, RTK GPS fix, altitude, and heading.
CameraWallModal/	Multi-camera surveillance grid for monitoring multiple live video links.
ReIDPanel/	Suspect gallery showing query crops and top matching aerial candidates.
TuningPanel/	Interactive sliders for adjusting AI confidence thresholds and zone polygons.
RecordingsView/	Filterable evidence database browser with video playback and photo preview modals.
IncidentLogs/	Real-time audit log of intrusion events with severity indicators.
dashboard/DroneDashboard.tsx	Master layout composing header, workspace body, and tactical overlays.

Directory: frontend/src/hooks/

Purpose: Custom React hooks abstracting state, side-effects, WebSockets, and audio. | **Tech:** React 19 Custom Hooks, Web Audio API, WebSocket API

File Name	Responsibility & Details
useDashboardOrchestrator.ts	Master orchestrator coordinating navigation tabs, URL routing, and camera modes.

<code>useTelemetrySocket.ts</code>	WebSocket connection to <code>`ws/telemetry`</code> streaming real-time bounding boxes into Redux.
<code>useAlertSocket.ts</code>	WebSocket listener for instant intrusion alarms and siren triggers.
<code>useReID.ts</code>	Polls Ground-to-Aerial Person Re-Identification status and query matches.
<code>useAudioAlert.ts</code>	Generates procedural audio sirens using Web Audio API on <code>`INTRUSION`</code> threat level.
<code>useCameraFleet.ts</code>	Queries and manages tactical cameras from <code>`/api/cameras`</code> .
<code>useCameraWall.ts</code>	Controls opening, closing, and selecting live camera feeds on the CCTV wall.
<code>useSystemClock.ts</code>	SSR hydration-safe UTC military clock synced across the entire interface.
<code>useZoneCanvas.ts</code>	Interactive HTML5 canvas logic for dragging and editing polygon geofence vertices.
<code>useVideoUpload.ts</code>	Handles uploading test <code>`mp4`</code> video files with progress percentage tracking.
<code>useDroneFlight.ts</code>	Dispatches drone flight commands to the backend API.
<code>useChannelPanel.ts</code>	Manages stream revisions, video replay triggers, and network camera streams.

Directory: frontend/src/services/

Purpose: Data fetching, Axios API clients, and TanStack React Query hooks. | **Tech:** Axios, TanStack React Query (v5)

File Name	Responsibility & Details
<code>api/client.ts</code>	Axios HTTP client configured with base URL <code>`/api`</code> , timeouts, and error handling.
<code>api/streamApi.ts</code>	API calls for switching feeds, testing connections, uploading videos, and replaying.
<code>api/cameraApi.ts</code>	CRUD API calls for registering and managing tactical cameras.
<code>api/reidApi.ts</code>	API calls for suspect matching status and retrieval retries.
<code>api/evidenceApi.ts</code>	Queries and deletes recorded media from the SQLite database.
<code>api/droneApi.ts</code>	Sends flight directives and camera health diagnostics requests.
<code>api/configApi.ts</code>	Fetches and updates AI detection thresholds and geofences.
<code>queries/useAlertsQuery.ts</code>	React Query hook fetching recent incident alerts with auto-polling.
<code>queries/useSnapshotsQuery.ts</code>	React Query hook caching the snapshot evidence gallery.
<code>queries/useStreamMutation.ts</code>	Mutations for switching perspective (<code>`aerial`</code> vs <code>`ground`</code>) and sources.

Directory: frontend/src/store/

Purpose: Redux Toolkit global state store. | **Tech:** Redux Toolkit, React Redux

File Name	Responsibility & Details
<code>index.ts</code>	Configures the Redux store and exports typed <code>`useAppSelector`</code> and <code>`useAppDispatch`</code> hooks.
<code>slices/telemetrySlice.ts</code>	Global state for live sensor metrics, active perspective, camera IDs, and layout mode.
<code>slices/uiSlice.ts</code>	Controls UI modals, suspect portraits, camera wall visibility, and audio mute states.
<code>slices/configSlice.ts</code>	Holds live AI confidence thresholds, IOU ratios, and restricted zone coordinates.

Directory: frontend/src/types/

Purpose: TypeScript interfaces, enums, and component prop contracts (Zero Inline Types rule). | **Tech:** TypeScript 5 Strict Mode

File Name	Responsibility & Details
-----------	--------------------------

common.ts	Core domain types: `ThreatLevel` (`CLEAR`, `MONITORING`, `INTRUSION`, `MANUAL`), `StreamSourceType`, `TacticalNavTab`.
telemetry.ts	Interfaces for live detections, bounding boxes, speed vectors, and drone avionics.
incidents.ts	Data models for incident alarms, threat categories, and timestamps.
reid.ts	Interfaces for Person Re-Identification queries, candidates, and similarity scores.
videoTesting.ts	Types for video uploading, suspect selection sessions, and playback clocks.
components/	Dedicated prop interfaces for every single tactical React component.
index.ts	Barrel export re-exporting all types from `@/types`.

Directory: frontend/src/utils/

Purpose: Pure mathematical helper functions, formatters, and style resolvers. | **Tech:** TypeScript Utilities

File Name	Responsibility & Details
threatUtils.ts	Resolves color tokens, badge backgrounds, and ribbon styles for each `ThreatLevel`.
formatters.ts	Formats UTC military timestamps (`HH:MM:SS UTC`), dates, file sizes, and telemetry.
canvasUtils.ts	Converts normalized polygon geofence points `[0.0, 1.0]` into canvas pixel coordinates.
apiError.ts	Extracts user-friendly error messages from Axios network exceptions.
cameraUtils.ts	Helper predicates for camera online statuses, IP stream validation, and perspective matching.
suspectSnapshots.ts	Generates safe download filenames and image blob representations for suspect evidence.
exportUtils.ts	Exports incident audit logs to CSV or JSON formats for offline security reviews.
gridUtils.ts	Calculates responsive split-screen viewport layout dimensions for multi-camera grids.

Directory: runs/output/

Purpose: Permanent forensic evidence storage directory. | **Tech:** SQLite Database, JPEG, MP4 H.264, JSON Logs

File Name	Responsibility & Details
evidence.db	SQLite database storing metadata for all 26,000+ saved evidence items with WAL mode.
snapshots/	Directory holding captured intruder snapshot photos (.jpg).
recordings/	Directory holding recorded surveillance video clips (.mp4).
logs/	Directory holding incident alarm history and telemetry log files.
backend/uploads/	Directory holding uploaded test surveillance videos (.mp4).